

Test Strategy

WebMind – AI Powered RAG Website Chatbot

Document Type	Test Strategy
Application Under Test	WebMind (RAG Website Chatbot)
Application URL	https://claysys-rag-project.vercel.app
Prepared As Part Of	QA Testing Intern - Technical Assignment (Pente AI)
Prepared By	Pradeepkumar M

1. Introduction & Purpose

This Test Strategy defines the overall approach for validating **WebMind**, a Retrieval-Augmented Generation (RAG) web application that allows users to convert any public website into a conversational, searchable knowledge base. The strategy establishes what will be tested, at what level, using which techniques, in what environment, and under what conditions testing is considered started (entry) and complete (exit).

This document is one deliverable within a broader QA Testing Intern technical assignment that also includes manual testing, exploratory testing, regression testing, unit testing, UI automation, end-to-end workflow automation, AI-specific testing, and API testing.

2. Project Scope

2.1 In Scope

- User authentication: Signup and Login flows (JWT-based sessions, password hashing)
- Dashboard: welcome screen, recent chats, knowledge base summary
- Knowledge Base creation: submitting a website URL, recursive crawling, chunking, and indexing
- Indexed Content viewer: page count, chunk count, vector count, per-page indexing status
- Conversational AI Q&A: asking questions, receiving RAG-generated answers
- Source citation / "Retrieved References" accuracy and traceability
- Grounded fallback behavior (system declining to answer when no relevant content is indexed)
- Multi-conversation and multi-knowledge-base management per user
- Session handling: login persistence, logout
- UI automation (Playwright) of login, one CRUD workflow, one AI interaction, and logout
- Unit tests (Jest & Pytest) for reusable/business-logic functions
- AI-specific testing: prompt validation, prompt injection, jailbreak attempts, guardrails, hallucination checks, context retention, multilingual and long/empty prompts
- API testing (if backend endpoints are accessible) via Postman

2.2 Out of Scope

- Load/performance testing at scale (e.g., hundreds of concurrent users) — not covered due to assignment time constraints
- Penetration testing / formal security audit beyond basic auth and input-validation checks
- Testing of third-party infrastructure reliability (Grok API uptime, Hugging Face Spaces uptime)
- PDF/document knowledge base ingestion — not a currently implemented feature
- Admin dashboard testing — feature does not currently exist
- Native mobile app testing — application is web-only

3. Test Levels

Level	Description	Applies To
Unit Testing	Validating individual functions/business logic in isolation	Reusable JS/Python functions (Jest, Pytest)
Integration Testing	Validating interaction between components (frontend <-> API, API <-> vector store <-> LLM)	Crawl-to-index pipeline, chat-to-answer pipeline
System Testing	End-to-end validation of complete features against requirements	Full manual test suite (Part 3)
Regression Testing	Re-validating critical flows after changes	Login, chat, indexing, citations (Part 5)
Acceptance / Exploratory Testing	Unscripted, experience-based testing to uncover unexpected defects	Full application (Part 4)
AI-Behavioral Testing	Testing model-specific behaviors unique to AI systems	Prompt handling, hallucination, guardrails (Part 9)

4. Test Types

Type	Focus
Functional Testing	Core features work as intended (signup, login, add website, chat, citations)
UI Testing	Layout, responsiveness, theme toggle, navigation, visual consistency
Validation Testing	Field-level input validation (email format, password rules, empty fields)
Negative Testing	Invalid inputs, wrong credentials, malformed URLs, unreachable websites
Boundary Testing	Very long URLs, very long questions, minimum/maximum password length, empty/near-empty websites
Authentication Testing	Signup/login correctness, session expiry, token handling, logout
Authorization Testing	Ensuring one user cannot access another user's knowledge bases/chats
Security Testing (basic)	SQL/NoSQL injection attempts, XSS in chat input, password hashing verification, JWT tampering
AI-Specific Testing	Prompt injection, jailbreak attempts, hallucination checks, guardrails, context retention, multilingual input
Regression Testing	Critical-path re-verification before release

Type	Focus
Automation Testing	Playwright UI automation and E2E business workflow automation
API Testing	Backend endpoint validation via Postman (if accessible)

5. Test Environment

Component	Details
Frontend	React + TypeScript + Vite, tested locally at localhost:5173
Backend	FastAPI (Python); live version hosted on Hugging Face Spaces
Vector Store	FAISS (per-website knowledge base index)
LLM Provider	Groq API
Browser(s)	Google Chrome (primary); cross-browser checks time-permitting
Automation Tooling	Playwright (JavaScript), Jest, Pytest, Postman
Test Data	Publicly accessible websites (e.g., www.python.org) used as crawl targets; dedicated test user account(s) for auth flows

6. Risks & Mitigations

Risk	Impact	Mitigation
Third-party LLM (Groq) latency/downtime during testing	Chat tests may fail for reasons outside the application's control	Log such failures separately from genuine application defects; retest before final reporting
Target crawl websites change content over time	Test cases referencing specific page counts/citations may become stale	Use a stable, low-change public site (e.g., python.org) as the primary crawl target
Limited time (32 hours) for an 11-part assignment	Some areas (e.g., load testing, deep security testing) cannot be fully covered	Prioritize core business workflow, authentication, and AI-specific testing first
No documented crawl depth/page limits	Boundary testing on crawler scope may be exploratory rather than exact	Treat undocumented limits as discovery-based exploratory testing and log findings as observations
Local vs. live environment differences	Defects found locally may not reproduce on the live Vercel/HF deployment or vice versa	Clearly note which environment each test was executed against

7. Assumptions

- Testing is primarily performed against the local development build (localhost:5173) unless stated otherwise
- A dedicated test user account is available and reusable across test cycles
- The Groq API and embedding services are available and within usage limits during testing
- The core business workflow (Add Website → Crawl/Index → Ask Question → Cited Answer) is the primary value path and receives the highest test priority
- "Ready for Release" criteria will be judged against this Test Strategy's exit criteria, not against a formally agreed SLA (none was provided)

8. Entry Criteria

- Application build is deployed and accessible on the test environment (local or live)
- Signup and Login pages are reachable without errors
- At least one website can be successfully submitted and begins indexing
- Test strategy, test cases, and test data are prepared and reviewed
- Required tooling (Playwright, Jest, Pytest, Postman) is installed and configured

9. Exit Criteria

- All planned test cases (manual, regression, exploratory) have been executed at least once
- No open Critical or High severity defects remain unresolved in the core business workflow (auth → add website → chat → citation)
- All Jest and Pytest unit tests pass (minimum 15 Jest / 10 Pytest as required by the assignment)
- Playwright automation suite (login, CRUD, AI interaction, logout) executes successfully end-to-end
- AI-specific test suite has been executed and findings (including any hallucination or guardrail gaps) are documented, even if not all are fixed
- All identified defects are logged in the Defect Report with severity, priority, and reproduction steps
- A Release Recommendation has been documented based on the above results

10. Roles & Responsibilities

Role	Responsibility
QA Engineer (Candidate)	Test planning, test case design, manual/exploratory/regression execution, automation development, defect reporting, release recommendation
Application Owner / Developer	Provide access to the application, clarify intended behavior, fix reported defects (outside this assignment's scope but relevant contextually)

11. Test Deliverables

- TestStrategy.pdf (this document)
- TestCases.xlsx – 75+ manual test cases
- DefectReport.xlsx – exploratory testing findings
- RegressionSuite.xlsx – 40+ regression scenarios
- UnitTesting/Jest and UnitTesting/Pytest – unit test suites
- Automation/ – Playwright UI automation framework
- BusinessWorkflowAutomation/ – end-to-end workflow automation
- AI-Testing/ – AI-specific test suite and findings
- Postman/ – API test collection (bonus)
- ReleaseRecommendation.pdf – final go/no-go recommendation